

Pseudocódigo com Análise de Complexidade

Complexidade e Computabilidade

Professor: Herriot

Projeto AV01

Grupo: Diego Feitosa, Diego Braga, José Kleyton, Lucas Espíndola

procedimento exibir_menu()

Linha de comando	Vezes executada (pior caso)
escreva("=====")	1
escreva("PROJETO AV01 - ESCOLHA UMA OPCAO")	1
escreva("=====")	1
escreva("1. Questao 1: Contagem de Ocorrencias")	1
escreva("2. Questao 2: Analise Triangular")	1
escreva("3. Questao 3: Matrices 3D")	1
escreva("4. Questao 4: Analise Assimetrica")	1
escreva("5. Questao 5: Busca Binaria")	1
escreva("0. Sair")	1
escreva("=====")	1
escreva("Digite sua escolha: ")	1
fim procedimento	

Expressão de complexidade: $T = 11$

Big O: $O(1)$

Cálculo de tempo: sem entrada $\rightarrow T = 11 = 11 \text{ instruções} \div 10^8 \text{ inst/s} = 1,1 \times 10^{-7} \text{ s}$

procedimento exibir_header()

Linha de comando	Vezes executada (pior caso)
escreva("=====")	1
escreva("COMPLEXIDADE E COMPUTABILIDADE")	1
escreva("PROFESSOR: HERRIOTR")	1
escreva("PROJETO AV01")	1
escreva("")	1
escreva("MEMBROS DO GRUPO:")	1
escreva("1. [Diego Feitosa]")	1
escreva("2. [Diego Braga]")	1
escreva("3. [Jose Kleyton]")	1
escreva("4. [Lucas Espindola]")	1
escreva("=====")	1
fim procedimento	

Expressão de complexidade: $T = 11$

Big O: $O(1)$

Cálculo de tempo: sem entrada $\rightarrow T = 11 = 11 \text{ instruções} \div 10^8 \text{ inst/s} = 1,1 \times 10^{-7} \text{ s}$

funcao principal()

Linha de comando	Vezes executada (pior caso)
<i>// Inicializa a semente para geração de números aleatórios</i>	
inicializar_gerador_aleatorio(tempo_atual)	1 (1 vez)
inteiro opcao	
chamar exibir_header()	11 (1 vez)
repita	
chamar exibir_menu()	11 (por iteração)
leia(opcao)	1

Linha de comando	Vezes executada (pior caso)
escolha (opcao)	1
caso 1:	
escreva("=== QUESTAO 1 ===")	1
chamar executar_q1()	T(executar_q1)
caso 2:	
escreva("=== QUESTAO 2 ===")	1
chamar executar_q2()	T(executar_q2)
caso 3:	
escreva("=== QUESTAO 3 ===")	1
chamar executar_q3()	T(executar_q3)
caso 4:	
escreva("=== QUESTAO 4 ===")	1
chamar executar_q4()	T(executar_q4)
caso 5:	
escreva("=== QUESTAO 5 ===")	1
chamar executar_q5()	T(executar_q5)
caso 0:	
escreva("Encerrando...")	1
caso contrario:	
escreva("Opcao invalida!")	1
fim escolha	
enquanto (opcao != 0)	1
retorne 0	1 (1 vez)
fim função	

Expressão de complexidade: $T = 13 + m \cdot (15 + T(\text{executar_qX}))$, $m = n^\circ$ de vezes que o menu é exibido (só um caso do escolha executa por iteração)

Big O: $O(1) + O(\text{executar_qX escolhida})$ o próprio menu é constante; o custo real está nas questões (ver abaixo)

Cálculo de tempo: 1 iteração, sem contar a questão $\rightarrow T = 13 + 15 = 28$ instruções $\div 10^8$ inst/s = $2,8 \times 10^{-7}$ s

funcao ler_opcao_entrada() retorna inteiro

Linha de comando	Vezes executada (pior caso)
inteiro opcao	
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")	1
leia(opcao)	1
retorne opcao	1
fim funcao	

Expressão de complexidade: $T = 3$

Big O: $O(1)$

Cálculo de tempo: sem entrada $\rightarrow T = 3 = 3$ instruções $\div 10^8$ inst/s = $3,0 \times 10^{-8}$ s

procedimento preencher_vetor_manual(vetor, inteiro tamanho)

Linha de comando	Vezes executada (pior caso)
para i de 0 ate tamanho - 1 faca	n+1
escreva("Digite o elemento [", i, "]: ")	n
leia(vetor[i])	n
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = (n+1) + n + n = 3n + 1$

Big O: $O(n)$

Cálculo de tempo: $n = 10^4 \trianglerightarrow T = 3 \cdot 10^4 + 1 = 30.001$ instruções $\div 10^8$ inst/s = $3,0 \times 10^{-4}$ s

procedimento preencher_vetor_aleatorio(vetor, inteiro tamanho)

Linha de comando	Vezez executada (pior caso)
para i de 0 ate tamanho - 1 faca	n+1
<i>// Gera um número aleatório entre 1 e 100</i>	
vetor[i] <- (aleatorio() mod 100) + 1	n
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = (n+1) + n = 2n + 1$

Big O: $O(n)$

Cálculo de tempo: $n = 10^4 \trianglerightarrow T = 2 \cdot 10^4 + 1 = 20.001 \text{ instruções} \div 10^8 \text{ inst/s} = 2,0 \times 10^{-4} \text{ s}$

procedimento exibir_vetor(vetor, inteiro tamanho, texto nome)

Linha de comando	Vezez executada (pior caso)
escreva(nome, " (tamanho ", tamanho, "):")	1
para i de 0 ate tamanho - 1 faca	n+1
escreva_mesma_linha(vetor[i], " ")	n
fim para	
escreva("") // Quebra de linha	1
fim procedimento	

Expressão de complexidade: $T(n) = 1 + (n+1) + n + 1 = 2n + 3$

Big O: $O(n)$

Cálculo de tempo: $n = 10^4 \trianglerightarrow T = 2 \cdot 10^4 + 3 = 20.003 \text{ instruções} \div 10^8 \text{ inst/s} = 2,0 \times 10^{-4} \text{ s}$

Questão 1:

procedimento contagem(inteiro n, inteiro k, vetorInteiro, listaElementos)

Linha de comando	Vezes executada (pior caso)
inteiro somaTotal <- 0	1
chamar exibir_vetor(vetorInteiro, n, "Vetor Principal")	2n+3
chamar exibir_vetor(listaElementos, k, "Vetor Buscados")	2k+3
escreva("RESULTADO:")	1
para i de 0 ate k - 1 faca	k+1
inteiro contagem elemento <- 0	k
para j de 0 ate n - 1 faca	k(n+1) = kn+k
se (vetorInteiro[j] == listaElementos[i]) entao	kn
contagem elemento <- contagem elemento + 1	kn (pior caso: todos iguais)
fim se	
fim para	
escreva("[", listaElementos[i], " aparece ", contagem elemento, " vez(es)]")	k
somaTotal <- somaTotal + contagem elemento	k
fim para	
escreva("Soma total: ", somaTotal)	1
fim procedimento	

Expressão de complexidade: $T(n,k) = 1 + (2n+3) + (2k+3) + 1 + (k+1) + k + (kn+k) + kn + kn + k + k + 1 = 3nk + 2n + 7k + 10$

Big O: $O(n \cdot k)$ (se $k \approx n$, $O(n^2)$)

Cálculo de tempo: $n = 10^4$, $k = 10^3 \trianglerightarrow T = 3 \cdot 10^4 \cdot 10^3 + 2 \cdot 10^4 + 7 \cdot 10^3 + 10 = 30.027.010$ instruções $\div 10^8$ inst/s = 0,3003 s

procedimento executar_q1()

Linha de comando	Vezes executada (pior caso)
inteiro n, k	
escreva("Digite o tamanho do vetor principal (n): ")	1
leia(n)	1
escreva("Digite a quantidade de buscados (k): ")	1
leia(k)	1
// Declaração dos vetores com os tamanhos informados	
inteiro vetorInteiro[n]	1
inteiro listaElementos[k]	1
inteiro opcao <- chamar ler_opcao_entrada()	3+1 = 4
se (opcao == 1) entao	1
escreva("Preenchendo vetor principal manualmente:")	1
chamar preencher_vetor_manual(vetorInteiro, n)	3n+1
escreva("Preenchendo vetor de buscados manualmente:")	1
chamar preencher_vetor_manual(listaElementos, k)	3k+1
senao	
chamar preencher_vetor_aleatorio(vetorInteiro, n)	2n+1 †
chamar preencher_vetor_aleatorio(listaElementos, k)	2k+1 †
escreva("Dados preenchidos aleatoriamente!")	1 †
fim se	
chamar contagem(n, k, vetorInteiro, listaElementos)	3nk+2n+7k+10
fim procedimento	

Expressão de complexidade: $T(n,k) = 6 + 4 + 1 + (3n+3k+4) + (3nk+2n+7k+10) = 3nk + 5n + 10k + 25$

Big O: $O(n \cdot k)$

Cálculo de tempo: $n = 10^4$, $k = 10^3 \trianglerightarrow T = 3 \cdot 10^7 + 5 \cdot 10^4 + 10 \cdot 10^3 + 25 = 30.060.025$ instruções $\div 10^8$ inst/s = 0,3006 s

Questão 2:

procedimento preencher_matriz_manual(inteiro n, inteiro m, matriz)

Linha de comando	Vezes executada (pior caso)
para i de 0 ate n - 1 faca	n+1
para j de 0 ate m - 1 faca	n(m+1)
escreva("Digite matriz[" , i, "][", j, "]: ")	nm
leia(matriz[i][j])	nm
fim para	
fim para	
fim procedimento	

Expressão de complexidade: $T(n,m) = (n+1) + n(m+1) + nm + nm = 3nm + 2n + 1$ (com $m = n$: $3n^2 + 2n + 1$)

Big O: $O(n \cdot m) \rightarrow O(n^2)$ quando $m = n$

Cálculo de tempo: $n = m = 10^3 \trianglerightarrow T = 3 \cdot 10^6 + 2 \cdot 10^3 + 1 = 3.002.001$ instruções $\div 10^8$ inst/s = 0,03 s

procedimento preencher_matriz_aleatorio(inteiro n, inteiro m, matriz)

Linha de comando	Vezes executada (pior caso)
para i de 0 ate n - 1 faca	n+1
para j de 0 ate m - 1 faca	n(m+1)
<i>// Gera um número aleatório entre 1 e 100</i>	
matriz[i][j] <- (aleatorio() mod 100) + 1	nm
fim para	
fim para	
fim procedimento	

Expressão de complexidade: $T(n,m) = (n+1) + n(m+1) + nm = 2nm + 2n + 1$ (com $m = n$: $2n^2 + 2n + 1$)

Big O: $O(n \cdot m) \rightarrow O(n^2)$ quando $m = n$

Cálculo de tempo: $n = m = 10^3 \trianglerightarrow T = 2 \cdot 10^6 + 2 \cdot 10^3 + 1 = 2.002.001$ instruções $\div 10^8$ inst/s = 0,02 s

procedimento exibir_matriz(inteiro n, inteiro m, matriz)

Linha de comando	Vezes executada (pior caso)
escreva("Matriz (", n, "x", m, "):")	1
para i de 0 ate n - 1 faca	n+1
para j de 0 ate m - 1 faca	n(m+1)
escreva_mesma_linha(matriz[i][j], " ")	nm
fim para	
escreva("") <i>// Quebra de linha para a próxima linha da matriz</i>	n
fim para	
fim procedimento	

Expressão de complexidade: $T(n,m) = 1 + (n+1) + n(m+1) + nm + n = 2nm + 3n + 2$ (com $m = n$: $2n^2 + 3n + 2$)

Big O: $O(n \cdot m) \rightarrow O(n^2)$ quando $m = n$

Cálculo de tempo: $n = m = 10^3 \trianglerightarrow T = 2 \cdot 10^6 + 3 \cdot 10^3 + 2 = 2.003.002$ instruções $\div 10^8$ inst/s = 0,02 s

funcao multiplo5(inteiro n, matriz) retorna inteiro

Linha de comando	Vezes executada (pior caso)
chamar exibir_matriz(n, n, matriz)	$2n^2 + 3n + 2$
inteiro mult5 <- 0	1
escreva("Verificacoes (i <= j):")	1
para i de 0 ate n - 1 faca	n+1
para j de 0 ate n - 1 faca	$n(n+1) = n^2 + n$
se (i <= j) entao	n^2
inteiro soma <- matriz[i][j] + matriz[j][i]	$n(n+1)/2$
<i>// O operador 'mod' retorna o resto da divisão</i>	
se (soma mod 5 == 0) entao	$n(n+1)/2$

Linha de comando	Vezes executada (pior caso)
escreva("matriz[" , i, "]" , j, "]" + matriz[" , j, "]" , i, "]" = " , matriz[i][j], " + " , matriz[j][i], " = " , soma, " (multiplo de 5)")	$n(n+1)/2$ (pior caso)
mult5 <- mult5 + 1	$n(n+1)/2$ (pior caso)
fim se	
fim se	
fim para	
fim para	
retorne mult5	1
fim funcao	

Expressão de complexidade: $T(n) = (2n^2+3n+2) + 1 + 1 + (n+1) + (n^2+n) + n^2 + 4 \cdot [n(n+1)/2] + 1 = 6n^2 + 7n + 6$

Big O: $O(n^2)$

Cálculo de tempo: $n = 10^3 \trianglerightarrow T = 6 \cdot 10^6 + 7 \cdot 10^3 + 6 = 6.007.006$ instruções $\div 10^8$ inst/s = 0,0601 s

procedimento executar_q2()

Linha de comando	Vezes executada (pior caso)
inteiro n, opcao, resultado	
escreva("Digite o tamanho da matriz (n x n): ")	1
leia(n)	1
// Declaração da matriz quadrada com dimensões n x n	
inteiro matriz[n][n]	1
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")	1
leia(opcao)	1
se (opcao == 1) entao	1
escreva("Preenchendo matriz manualmente:")	1
chamar preencher_matriz_manual(n, n, matriz)	$3n^2+2n+1$
senao	
chamar preencher_matriz_aleatorio(n, n, matriz)	$2n^2+2n+1$ †
escreva("Matriz preenchida aleatoriamente!")	1 †
fim se	
resultado <- chamar multiplo5(n, matriz)	$6n^2+7n+6 + 1$
escreva("RESULTADO:")	1
escreva("Total de multiplos de 5: " , resultado)	1
fim procedimento	

Expressão de complexidade: $T(n) = 5 + 1 + (3n^2+2n+2) + (6n^2+7n+7) + 2 = 9n^2 + 9n + 17$

Big O: $O(n^2)$

Cálculo de tempo: $n = 10^3 \trianglerightarrow T = 9 \cdot 10^6 + 9 \cdot 10^3 + 17 = 9.009.017$ instruções $\div 10^8$ inst/s = 0,0901 s

Questão 3:

procedimento preencher_matriz3d_manual(inteiro n, matriz)

Linha de comando	Vezes executada (pior caso)
para i de 0 ate n - 1 faca	$n+1$
para j de 0 ate n - 1 faca	$n(n+1) = n^2+n$
para k de 0 ate n - 1 faca	$n^2(n+1) = n^3+n^2$
escreva("Digite matriz[" , i, "][", j, "][", k, "]: ")	n^3
leia(matriz[i][j][k])	n^3
fim para	
fim para	
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = (n+1) + (n^2+n) + (n^3+n^2) + n^3 + n^3 = 3n^3 + 2n^2 + 2n + 1$

Big O: $O(n^3)$

Cálculo de tempo: $n = 100 \trianglerightarrow T = 3 \cdot 10^6 + 2 \cdot 10^4 + 200 + 1 = 3.020.201$ instruções $\div 10^8$ inst/s = 0,0302 s

procedimento preencher_matriz3d_aleatorio(inteiro n, matriz)

Linha de comando	Vezes executada (pior caso)
para i de 0 ate n - 1 faca	$n+1$
para j de 0 ate n - 1 faca	n^2+n
para k de 0 ate n - 1 faca	n^3+n^2
matriz[i][j][k] <- (aleatorio() mod 100) + 1	n^3
fim para	
fim para	
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = (n+1) + (n^2+n) + (n^3+n^2) + n^3 = 2n^3 + 2n^2 + 2n + 1$

Big O: $O(n^3)$

Cálculo de tempo: $n = 100 \trianglerightarrow T = 2 \cdot 10^6 + 2 \cdot 10^4 + 200 + 1 = 2.020.201$ instruções $\div 10^8$ inst/s = 0,0202 s

procedimento exibir_matriz3d_resumido(inteiro n, matriz)

Linha de comando	Vezes executada (pior caso)
escreva("Matriz (, n, "x", n, "x", n, ") - Primeiros elementos:")	1
// Define o limite de camadas a exibir	
inteiro limite_camadas	
se (n < 2) entao	1
limite_camadas <- n	1 †
senao	
limite_camadas <- 2	1 †
fim se	
para i de 0 ate limite_camadas - 1 faca	3 ($n \geq 2 \rightarrow 2$ camadas)
escreva("Camada " , i, ":")	2
para j de 0 ate n - 1 faca	$2(n+1)$
para k de 0 ate n - 1 faca	$2n(n+1)$
escreva_mesma_linha(matriz[i][j][k], " ")	$2n^2$
fim para	
escreva("") // Quebra de linha para a linha da matriz	2n
fim para	
escreva("") // Quebra de linha para separar as camadas	2
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = 1 + 2 + 3 + 2 + (2n+2) + (2n^2+2n) + 2n^2 + 2n + 2 = 4n^2 + 6n + 12 \quad (n \geq 2)$

Big O: $O(n^2)$

Cálculo de tempo: $n = 100 \trianglerightarrow T = 4 \cdot 10^4 + 600 + 12 = 40.612 \text{ instruções} \div 10^8 \text{ inst/s} = 4,1 \times 10^{-4} \text{ s}$

funcao comparaMatrizes3D(inteiro n, matrizA, matrizB) retorna inteiro

Linha de comando	Vezes executada (pior caso)
chamar exibir_matriz3d_resumido(n, matrizA)	$4n^2+6n+12$
<i>// Usando inteiro_longo para evitar overflow em somas grandes</i>	
inteiro_longo somaA <- 0	1
inteiro_longo somaB <- 0	1
para i de 0 ate n - 1 faca	$n+1$
para j de 0 ate n - 1 faca	n^2+n
para k de 0 ate n - 1 faca	n^3+n^2
somaA <- somaA + matrizA[i][j][k]	n^3
somaB <- somaB + matrizB[i][j][k]	n^3
fim para	
fim para	
fim para	
escreva("RESULTADO:")	1
escreva("Soma A: ", somaA)	1
escreva("Soma B: ", somaB)	1
se (somaA >= somaB) entao	1
escreva("A >= B? SIM (retorna 1)")	1
retorne 1	1
senao	
escreva("A >= B? NAO (retorna 0)")	1 †
retorne 0	1 †
fim se	
fim funcao	

Expressão de complexidade: $T(n) = (4n^2+6n+12) + 2 + (n+1) + (n^2+n) + (n^3+n^2) + 2n^3 + 3 + 1 + 2 = 3n^3 + 6n^2 + 8n + 21$

Big O: $O(n^3)$

Cálculo de tempo: $n = 100 \trianglerightarrow T = 3 \cdot 10^6 + 6 \cdot 10^4 + 800 + 21 = 3.060.821 \text{ instruções} \div 10^8 \text{ inst/s} = 0,0306 \text{ s}$

procedimento executar_q3()

Linha de comando	Vezes executada (pior caso)
inteiro n, opcao	
escreva("Digite o tamanho da matriz (n x n x n): ")	1
leia(n)	1
<i>// Declaração das matrizes tridimensionais</i>	
inteiro matrizA[n][n][n]	1
inteiro matrizB[n][n][n]	1
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")	1
leia(opcao)	1
se (opcao == 1) entao	1
escreva("Preenchendo matriz A manualmente:")	1
chamar preencher_matriz3d_manual(n, matrizA)	$3n^3+2n^2+2n+1$
escreva("Preenchendo matriz B manualmente:")	1
chamar preencher_matriz3d_manual(n, matrizB)	$3n^3+2n^2+2n+1$
senao	
chamar preencher_matriz3d_aleatorio(n, matrizA)	$2n^3+2n^2+2n+1 \dagger$
chamar preencher_matriz3d_aleatorio(n, matrizB)	$2n^3+2n^2+2n+1 \dagger$
escreva("Matrizes preenchidas aleatoriamente!")	1 †
fim se	
chamar comparaMatrizes3D(n, matrizA, matrizB)	$3n^3+6n^2+8n+21$
fim procedimento	

Expressão de complexidade: $T(n) = 6 + 1 + (6n^3 + 4n^2 + 4n + 4) + (3n^3 + 6n^2 + 8n + 21) = 9n^3 + 10n^2 + 12n + 32$

Big O: $O(n^3)$

Cálculo de tempo: $n = 100 \trianglerightarrow T = 9 \cdot 10^6 + 10^5 + 1200 + 32 = 9.101.232 \text{ instruções} \div 10^8 \text{ inst/s} = 0,091 \text{ s}$

Questão 4:

funcao calcular_fatorial(inteiro n) retorna inteiro_longo

Linha de comando	Vezes executada (pior caso)
se (n <= 1) entao	1
retorne 1	1 († só se $n \leq 1$)
fim se	
inteiro longo fat <- 1	1
para i de 2 ate n faca	n
fat <- fat * i	n-1
fim para	
retorne fat	1
fim funcao	

Expressão de complexidade: $T(n) = 1 + 1 + n + (n-1) + 1 = 2n + 2$ ($n = \text{valor recebido}, n > 1$)

Big O: $O(n)$

Cálculo de tempo: $n = 15 \trianglerightarrow (\text{máximo do modo aleatório}) \rightarrow T = 2 \cdot 15 + 2 = 32 \text{ instruções} \div 10^8 \text{ inst/s} = 3,2 \times 10^{-7} \text{ s}$

funcao processar_vetor(inteiro n, vetor) retorna inteiro_longo

Linha de comando	Vezes executada (pior caso)
escreva("Vetor (tamanho ", n, "):")	1
// Define limite de exibição para não poluir a tela	
inteiro limite_exibicao	
se (n < 50) entao	1
limite_exibicao <- n	1 †
senao	
limite_exibicao <- 50	1 †
fim se	
para i de 0 ate limite_exibicao - 1 faca	51 ($n \geq 50$)
escreva_mesma_linha(vetor[i], " ")	50
fim para	
se (n > 50) entao	1
escreva_mesma_linha("...")	1
fim se	
escreva("") // Quebra de linha	1
escreva("Processamento:")	1
inteiro longo somatorio <- 0	1
para i de 0 ate n - 1 faca	n+1
se (vetor[i] mod 2 == 0) entao	n
escreva("vetor[", i, "] = ", vetor[i], " (PAR) -> soma: +", vetor[i])	n †
somatorio <- somatorio + vetor[i]	n †
senao	
inteiro longo fat <- chamar calcular_fatorial(vetor[i])	n(2m+2) + n
escreva("vetor[", i, "] = ", vetor[i], " (IMPAR) -> fatorial: ", fat)	n
somatorio <- somatorio + fat	n
fim se	
fim para	
retorne somatorio	1
fim funcao	

Expressão de complexidade: $T(n) = 1 + 2 + 51 + 50 + 2 + 3 + (n+1) + n + n(2m+3) + n + n + 1 = 2nm + 7n + 111$ ($n \geq 50$; m = maior valor do vetor; pior caso: todos ímpares)

Big O: $O(n \cdot m)$. No modo aleatório $m \leq 15$ (constante) $\rightarrow O(n)$

Cálculo de tempo: $n = 10^4$, $m = 15 \trianglerightarrow T = 2 \cdot 10^4 \cdot 15 + 7 \cdot 10^4 + 111 = 370.111$ instruções $\div 10^8$ inst/s = 0,0037 s

procedimento executar_q4()

Linha de comando	Vezes executada (pior caso)
inteiro n, opcao	
escreva("Digite o tamanho do vetor (n): ")	1
leia(n)	1
inteiro vetor[n]	1
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")	1
leia(opcao)	1
se (opcao == 1) entao	1
escreva("Preenchendo vetor manualmente:")	1
para i de 0 ate n - 1 faca	n+1
escreva("Digite o elemento [", i, "]: ")	n
leia(vetor[i])	n
fim para	
senao	
para i de 0 ate n - 1 faca	n+1 †
// Valores entre 1 e 15 para evitar que o fatorial passe do limite	
vetor[i] <- (aleatorio() mod 15) + 1	n †
fim para	
escreva("Vetor preenchido aleatoriamente!")	1 †
fim se	
inteiro_longo resultado <- chamar processar_vetor(n, vetor)	2nm+7n+111 + 1
escreva("RESULTADO:")	1
escreva("Somatorio: ", resultado)	1
fim procedimento	

Expressão de complexidade: $T(n) = 5 + 1 + (3n+2) + (2nm+7n+112) + 2 = 2nm + 10n + 122$

Big O: $O(n \cdot m)$ (modo aleatório: $O(n)$)

Cálculo de tempo: $n = 10^4$, $m = 15 \trianglerightarrow T = 3 \cdot 10^5 + 10^5 + 122 = 400.122$ instruções $\div 10^8$ inst/s = 0,004 s

Questão 5:

funcao busca_binaria(inteiro n, vetor, inteiro alvo) retorna inteiro

Linha de comando	Vezes executada (pior caso)
inteiro inicio <- 0	1
inteiro fim <- n - 1	1
enquanto (inicio <= fim) faca	$\log n + 1$
inteiro meio <- inicio + (fim - inicio) / 2	$\log n$
se (vetor[meio] == alvo) entao	$\log n$
retorne 1	1 († só se achar; pior caso: não acha)
fim se	
se (vetor[meio] < alvo) entao	$\log n$
inicio <- meio + 1	$\log n + 1$
senao	
fim <- meio - 1	$\log n + 1$
fim se	
fim enquanto	
retorne 0	1
fim funcao	

Expressão de complexidade: $T(n) = 1 + 1 + (\log n + 1) + \log n + \log n + \log n + \log n + 1 = 5 \cdot \log n + 4$ ($\log = \log_2$; pior caso: alvo ausente)

Big O: $O(\log n)$

Cálculo de tempo: $n = 10^4 \triangle (\log_2 n \approx 13,29) \rightarrow T = 5 \cdot 13,29 + 4 = 70 \text{ instruções} \div 10^8 \text{ inst/s} = 7,0 \times 10^{-7} \text{ s}$

procedimento ordenar_vetor(vetor, inteiro n)

Linha de comando	Vezes executada (pior caso)
<i>// Implementação do algoritmo Bubble Sort</i>	
para i de 0 ate n - 2 faca	n
para j de 0 ate n - i - 2 faca	$\Sigma(n-i) = n(n+1)/2 - 1$
se (vetor[j] > vetor[j+1]) entao	$n(n-1)/2$
inteiro temp <- vetor[j]	$n(n-1)/2$ (pior caso)
vetor[j] <- vetor[j+1]	$n(n-1)/2$
vetor[j+1] <- temp	$n(n-1)/2$
fim se	
fim para	
fim para	
fim procedimento	

Expressão de complexidade: $T(n) = n + [n(n+1)/2 - 1] + 4 \cdot [n(n-1)/2] = (5n^2 - n - 2)/2$ (pior caso: vetor decrescente)

Big O: $O(n^2)$

Cálculo de tempo: $n = 10^4 \triangle \rightarrow T = (5 \cdot 10^8 - 10^4 - 2)/2 = 249.994.999 \text{ instruções} \div 10^8 \text{ inst/s} = 2,4999 \text{ s}$

funcao contar_elementos(inteiro n, vetorA, vetorB) retorna inteiro

Linha de comando	Vezes executada (pior caso)
escreva("Vetor A (nao ordenado) - primeiros 20 elementos:")	1
inteiro limite_exibicao	
se (n < 20) entao	1
limite_exibicao <- n	1 †
senao	
limite_exibicao <- 20	1 †
fim se	
para i de 0 ate limite_exibicao - 1 faca	21 (n ≥ 20)
escreva_mesma_linha(vetorA[i], " ")	20
fim para	
se (n > 20) entao	1

Linha de comando	Vezes executada (pior caso)
escreva_mesma_linha("...")	1
fim se	
escreva("") // Quebra de linha	1
escreva("Vetor B (ordenado) - primeiros 20 elementos:")	1
para i de 0 ate limite_exibicao - 1 faca	21
escreva_mesma_linha(vetorB[i], " ")	20
fim para	
se (n > 20) entao	1
escreva_mesma_linha("...")	1
fim se	
escreva("") // Quebra de linha	1
escreva("Processando...")	1
inteiro total_encontrados <- 0	1
para i de 0 ate n - 1 faca	n+1
// Se a busca binária retornar 1, incrementa o total	
se (chamar_busca_binaria(n, vetorB, vetorA[i]) == 1) entao	n · (1 + 5 · log n + 4)
total_encontrados <- total_encontrados + 1	n (pior caso)
fim se	
fim para	
retorne total_encontrados	1
fim funcao	

Expressão de complexidade: $T(n) = 96 + n + n(5 \cdot \log n + 5) + n = 5n \cdot \log n + 7n + 96 \quad (n \geq 20)$

Big O: $O(n \log n)$

Cálculo de tempo: $n = 10^4 \trianglerightarrow T = 5 \cdot 10^4 \cdot 13,29 + 7 \cdot 10^4 + 96 = 734.482 \text{ instruções} \div 10^8 \text{ inst/s} = 0,0073 \text{ s}$

procedimento executar_q5()

Linha de comando	Vezes executada (pior caso)
inteiro n, opcao	
escreva("Digite o tamanho dos vetores (n): ")	1
leia(n)	1
inteiro vetorA[n]	1
inteiro vetorB[n]	1
escreva("Deseja entrada manual (1) ou aleatoria (2)? ")	1
leia(opcao)	1
se (opcao == 1) entao	1
escreva("Preenchendo vetor A manualmente:")	1 †
para i de 0 ate n - 1 faca	n+1 †
escreva("Digite o elemento [", i, "]: ")	n †
leia(vetorA[i])	n †
fim para	
escreva("Preenchendo vetor B manualmente (DEVE ESTAR ORDENADO):")	1 †
para i de 0 ate n - 1 faca	n+1 †
escreva("Digite o elemento [", i, "]: ")	n †
leia(vetorB[i])	n †
fim para	
senao	
para i de 0 ate n - 1 faca	n+1
vetorA[i] <- (aleatorio() mod 200) + 1	n
vetorB[i] <- (aleatorio() mod 200) + 1	n
fim para	
// Garante que o vetor B esteja ordenado	
chamar ordenar_vetor(vetorB, n)	$(5n^2 - n - 2)/2$
escreva("Vetores preenchidos aleatoriamente! Vetor B foi ordenado.")	1
fim se	
inteiro resultado <- chamar contar_elementos(n, vetorA, vetorB)	$5n \cdot \log n + 7n + 96 + 1$
escreva("RESULTADO:")	1

Linha de comando	Vezes executada (pior caso)
escreva("Total de elementos de A encontrados em B: ", resultado)	1
fim procedimento	

Expressão de complexidade: $T(n) = 6 + 1 + [(3n+1) + (5n^2-n-2)/2 + 1] + (5n \cdot \log n + 7n + 97) + 2 = 2,5n^2 + 5n \cdot \log n + 9,5n + 107$ (o ramo aleatório, com Bubble Sort, é o pior)

Big O: $O(n^2)$ dominado pelo ordenar_vetor (a busca em si é $O(n \log n)$)

Cálculo de tempo: $n = 10^4 \trianglerightarrow T = 2,5 \cdot 10^8 + 5 \cdot 10^4 \cdot 13,29 + 9,5 \cdot 10^4 + 107 = 250.759.493$ instruções $\div 10^8$ inst/s = 2,5076 s